

Week 2 Task: Cloud Automation and Infrastructure as Code Implementation

Terraform-based AWS implementation for a multi-tier e-commerce application

Important: This report does not claim that AWS resources were actually deployed. The evidence section is a checklist for capturing genuine deployment outputs after the student runs Terraform.

1. Objective

Automate the creation, modification and management of cloud resources using Infrastructure as Code. Terraform is used to define an AWS environment containing networking, a load-balanced web tier, a database, auto-scaling and monitoring.

2. Architecture and Components

The implementation creates a VPC across two Availability Zones, public subnets for the Application Load Balancer, private subnets for EC2 application instances and RDS, an Auto Scaling Group, an RDS MySQL database, an S3 bucket, IAM/Systems Manager access and a CloudWatch alarm.

Internet → Application Load Balancer → EC2 Auto Scaling Group → private RDS MySQL. Application instances can also use S3 for assets/artifacts.

3. Networking and Security

The ALB security group allows HTTP/HTTPS from the internet. The application security group allows HTTP only from the ALB security group. The database security group allows TCP/3306 only from the application security group. RDS is private and is not publicly accessible. EC2 uses Systems Manager instead of opening SSH.

4. Infrastructure as Code

The Terraform project is split into provider/version configuration, variables, main infrastructure resources and example variable values. Dependencies are expressed through resource references, allowing Terraform to build resources in the correct order.

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

5. Auto-scaling

The application Auto Scaling Group starts with two instances, can scale to six, and uses target tracking at 60% average CPU. This is a demonstration policy; production thresholds should be validated with load tests and application latency measurements.

6. Monitoring

A CloudWatch alarm monitors elevated ALB 5xx responses. A production system should additionally monitor latency, target health, application errors, CPU/memory, RDS connections, storage, replication and billing.

7. Database and Cost Controls

The lab uses a small RDS instance and disables Multi-AZ to control cost. This is deliberately a development configuration, not a production recommendation. Production should enable Multi-AZ, stronger backup/DR and appropriate sizing.

8. Dependencies / Prerequisites

Required: AWS account and permissions, AWS CLI credentials, Terraform ≥ 1.6 and internet access for Terraform provider/module downloads. The database password must be supplied securely and must not be committed to source control.

9. Validation and Evidence

The assignment requests screenshots or terminal output. These must come from an actual deployment. The supplied `DEPLOYMENT_EVIDENCE.md` gives a capture checklist for terraform validate, plan, apply, outputs, AWS Console resources, application response and scaling.

10. Post-deployment Analysis

Terraform reduces manual provisioning and makes the infrastructure reproducible. The trade-off is that infrastructure code becomes production code: state files need protection, changes require review, provider versions should be controlled, and drift must be detected. CI/CD should run formatting, validation, security checks and policy checks before apply.

11. Limitations and Production Improvements

The lab does not implement HTTPS certificates, WAF, RDS Multi-AZ, remote encrypted Terraform state, a real application container, or a multi-region disaster-recovery design. These are intentional scope/cost limitations and are documented in `README.md`.

12. Conclusion

The project demonstrates a complete IaC workflow covering networking, compute, load balancing, database, security, auto-scaling and monitoring. The infrastructure is modular enough to extend into a production-oriented environment, provided the documented security, state-management and availability upgrades are implemented.